VINUNIVERSITY

Monitoring, Logging & Observability

AICB-P1 · Ngày 13 · Biết agent đang chạy thế nào trước khi user phàn nàn

Tên Giảng Viên

VinUniversity · Phase 1 · 2026

HÃY SUY NGHĨ...

“Agent bạn deploy hôm Day 12 chạy ngon. 3 ngày sau: latency tăng gấp đôi, cost tăng 300 phần trăm, và 1 trên 20 câu trả lời là bịa. Bạn biết những điều này khi nào? — Khi user phàn nàn. Đó là cách tệ nhất, và đắt nhất, để phát hiện vấn đề.”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội Dung Bài Học

1. Vì sao agent cần observability
2. 3 pillars + pillar thứ 4
3. AI-specific metrics
4. Structured logging
5. Distributed tracing cho agent
6. Bộ công cụ LLM-observability 2026
7. Production stack: Prometheus + Grafana
8. Dashboard design
9. Alerting & SLO
10. Cost monitoring & optimization
11. Debug 1 incident bằng trace
12. Human feedback & online eval
13. Privacy & compliance khi logging
14. Checklist, Lab 13 & tổng kết

Mục Tiêu Ngày 13 (Bloom's Taxonomy)

- **Remember** — liệt kê **3 pillars** (metrics, logs, traces) + **pillar thứ 4** (continuous eval) và 6 nhóm AI-specific metrics
- **Understand** — giải thích vì sao **P99 quan trọng hơn average**, và SLO khác SLA thế nào
- **Apply** — implement **structured logging** (JSON + correlation ID + PII redaction) cho agent đang deploy
- **Analyze** — đọc **trace waterfall** và xác định bottleneck trong agent pipeline nhiều bước
- **Evaluate** — so sánh Langfuse / LangSmith / Phoenix / Helicone cho use case cụ thể
- **Create** — thiết kế **monitoring blueprint** với SLO, alert rules (symptom-based) và dashboard 3 layers

Deliverable Cuối Ngày

Agent có observability đầy đủ: bạn biết nó chạy thế nào mà **không cần hỏi user**.

- Structured logging pipeline: JSON, correlation ID, input/output đã **redact PII**
- Tracing: Langfuse (hoặc backend zero-key) connected, ≥ 10 traces
- Dashboard: latency P50/95/99 + TTFT, cost/ngày, error rate, token usage, tool-call success
- ≥ 3 alert rules $\rightarrow$ Slack; 1 SLO + error budget; 1 incident note đọc từ trace

Vì Sao Agent Cần Observability

01

“It works” không đủ cho production — cần biết nó chạy TỐT đến đâu, chậm ở đâu, tốn bao nhiêu, và khi nào sắp hỏng

Từ Artifact Day 12 Sang Production Reality

Day 12 đã làm được

- Agent deployed trên cloud
- Public URL hoạt động
- Health check endpoint
- Basic authentication

Nhưng chưa trả lời được

- Agent đang chậm hay nhanh?
- Tốn bao nhiêu tiền mỗi ngày?
- Bao nhiêu request fail (hoặc trả lời sai)?
- Khi nào cần scale up?

Lưu ý: Không có monitoring, bạn chỉ biết agent hỏng **khi user phàn nàn**. Health check "200 OK" không có nghĩa câu trả lời đúng.

Monitoring vs Observability — Hai Khái Niệm Khác Nhau

VINUNIVERSITY

Monitoring

Theo dõi các câu hỏi **đã biết trước**.

- Dashboard + alert dựng sẵn
- Trả lời: "**X có hỏng không?**"
- Tốt cho failure mode đã lường trước
- "Known-knowns"

Observability

Thuộc tính của hệ thống: hỏi câu **mới** mà không cần deploy code.

- Telemetry đủ giàu (metrics + logs + traces)
- Trả lời: "**TẠI SAO X hỏng?**"
- Tốt cho failure mode **chưa từng gặp**
- "Unknown-unknowns"

Observability AI Khác Gì Monitoring Phần Mềm Truyền Thống?

Cùng input, output khác nhau mỗi lần. Không thể test bằng "so sánh string". Phải đo **chất lượng**, không chỉ pass/fail.

Mỗi request tốn tiền theo số token. Một bug loop có thể đốt budget trong vài giờ — CPU/RAM không nói cho bạn biết.

App không "crash" — nó vẫn trả 200 OK nhưng câu trả lời tệ dần. Không có exception để bắt.

Hallucinated tool args, vòng lặp vô tận, context overflow, prompt injection — những lỗi mà APM truyền thống không có khái niệm.

Cost of Not Monitoring

Agent trả lời sai nhưng không ai biết. User mất niềm tin dần. Đến khi phát hiện thì đã mất user.

Token cost tăng dần mà không alert. Cuối tháng nhận bill gấp 5 lần. Đốt hết budget trước khi kịp react.

Latency P95 tăng 10ms mỗi tuần. 6 tuần sau: chậm gấp đôi. Không ai để ý vì không có baseline.

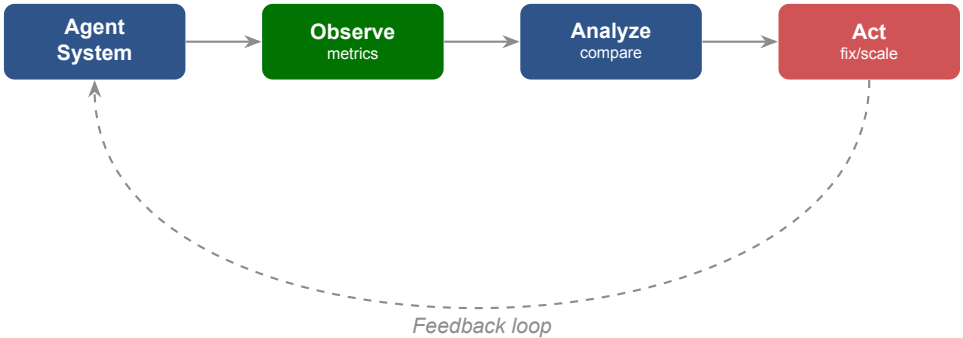
Bug report: “agent sai hôm qua.” Không log, không trace. Không reproduce được. Không fix được.

Observability: Vài Cột Mốc



1. Logs (text)
2. Metrics & Prometheus 2012
3. Grafana 2014
4. Tracing & OTel 2019
5. LLM-native 2023+

Control Theory — Observability Là Một Feedback Loop



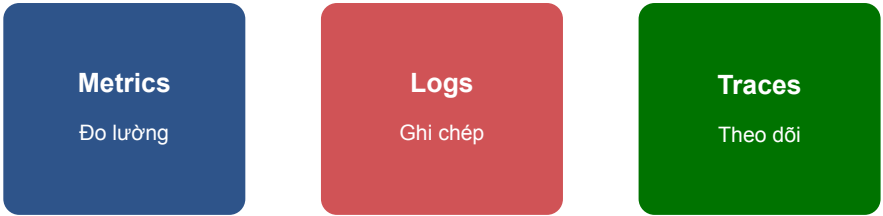
Mean Time To Detect: từ khi sự cố xảy ra đến khi phát hiện.

Mean Time To Recover: từ khi phát hiện đến khi fix xong.

3 Pillars + Pillar Thứ 4

Metrics nói bao nhiêu / bao lâu, logs nói chuyện gì xảy ra, traces nói tại sao — và với AI có pillar thứ 4: câu trả lời có còn ĐÚNG không

3 Pillars of Observability



Logs nói chuyện gì xảy ra. **Metrics** nói bao nhiêu và bao lâu. **Traces** nói tại sao.

Pillar Thứ 4: Continuous / Online Eval

VINUNIVERSITY

Pillar thứ 4 — Với AI system, ba pillar truyền thống không trả lời được câu hỏi quan trọng nhất: **câu trả lời có còn đúng không?** Pillar thứ 4 = đo **chất lượng output** liên tục trên production.

- HTTP 200 **khác** correctness — request "thành công" vẫn có thể là câu trả lời bịa
- Latency thấp **khác** hữu ích — trả lời nhanh nhưng sai còn tệ hơn
- Error rate 0% **khác** không đốt tiền — chi phí vẫn có thể tăng vọt

Day 13 đo chất lượng **liên tục trên production** (online). Day 14 đo chất lượng **có hệ thống bằng benchmark** (offline). Cả hai bổ sung cho nhau.

Tại Sao Chỉ Logs Là Không Đủ?

Chỉ có logs

- biết request nào fail
- nhưng không biết fail rate bao nhiêu
- không biết latency đang tăng dần
- không biết bottleneck ở đâu

Đủ pillars

- metrics cho biết trend (tăng/giảm)
- logs cho biết chi tiết từng request
- traces cho biết chậm ở bước nào
- eval cho biết chất lượng còn tốt không

Logs giống camera an ninh. Metrics giống bảng điều khiển xe. Traces giống bản đồ GPS. Eval giống người kiểm định chất lượng. Cần cả bốn để **lái an toàn**.

Pillar Nào Trả Lời Câu Hỏi Nào?

Câu hỏi	Pillar	Công cụ ví dụ
“Error rate có tăng không?”	Metrics	Prometheus, Grafana
“Request `req-abc` đã làm gì?”	Logs	Loki, JSON logs
“Chậm ở bước nào trong agent loop?”	Traces	Langfuse, Tempo
“Câu trả lời còn đúng không?”	Eval (4th)	LLM-judge, RAGAS

Chọn pillar theo câu hỏi bạn cần trả lời. Đừng thu thập telemetry chỉ vì có thể — mỗi data point đều tốn tiền lưu trữ (xem §10).

AI-Specific Metrics

03

Monitoring truyền thống đo CPU, RAM, uptime — AI agent cần thêm: token, cost, TTFT, chất lượng, tool-call success, retrieval quality

4 Nhóm Metrics Cho AI Agent

VINUNIVERSITY

Performance

- Latency P50 / P95 / P99
- Time to first token (TTFT)
- Throughput (req/s, tokens/s)
- LLM call duration

Cost

- Tokens per request (in / out)
- Cost per request / per task
- Cost per day / per user / per feature
- Cache hit rate

Quality (pillar 4)

- Hallucination / faithfulness
- Task completion rate
- Thumbs up/down, regenerate rate
- Guardrail trigger rate

Reliability

- Error rate, uptime
- Tool-call success / failure rate
- Retry rate, loop rate
- Retrieval recall / empty-result rate

4 Golden Signals + 2 Cho AI Agent

VINUNIVERSITY

Google SRE — 4 Golden Signals

1. **Latency** — thời gian phản hồi
2. **Traffic** — request rate (QPS)
3. **Errors** — error rate
4. **Saturation** — tài nguyên còn bao nhiêu

AI agent cần thêm 2

5. **Cost** — $/request, $/user, token usage
6. **Quality** — hallucination rate, CSAT, groundedness

Lưu ý: Agent có thể “up” (traffic/latency/error OK) nhưng **trả lời sai và đốt tiền**. Đây là 2 failure mode riêng của AI mà monitoring truyền thống bỏ qua.

RED vs USE — Hai Phương Pháp Observability

VINUNIVERSITY

RED (request-centric)

- **R**ate — requests/giây
- **E**rrors — error rate
- **D**uration — latency P50/P95/P99

Góc nhìn user: tôi gửi request, được gì?

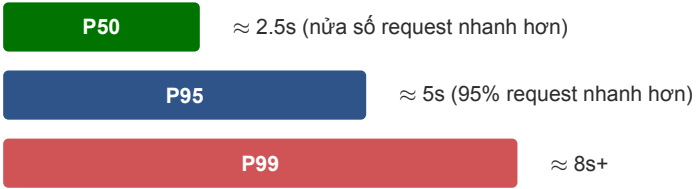
USE (resource-centric)

- **U**tilization — tài nguyên dùng %
- **S**aturation — có queue/chờ không?
- **E**rrors — lỗi của resource

Góc nhìn resource: LLM API, queue đang làm gì?

Agent chậm (RED: Duration P95 tăng) → debug bằng USE (LLM rate-limit utilization 95%) → bị throttle → upgrade tier hoặc fallback.

Latency: Percentiles + TTFT



TTFT (Time To First Token) — Thời gian từ lúc gửi request đến token đầu tiên. Quyết định cảm giác "nhanh". Điển hình 2026: P50 ≈ 0.5–1.0s, P95 ≈ 1.5–2.5s.

Lưu ý: Trung bình (average) ẩn long tail. P95 mới là trải nghiệm thật. **Reasoning mode** là lớp latency riêng (chậm hơn 5–30x) — tách ra khi đo.

Percentile Math — Vì Sao P99 Quan Trọng

Bài toán — Agent có P99 = 5s, user chat 10 lượt. Xác suất gặp ≥ 1 lần > 5s:

$$P = 1 - 0.99^{10} \approx 9.6\%$$

"Every 100ms of latency cost 1% of sales." → optimize **tail**, không chỉ average. P99 / P99.9 là KPI chính thức tại Amazon, Google, Meta.

- 1/10 user sẽ gặp lag rất tệ
- 1.000 user/ngày → **96 user** bức xúc
- Họ là người tweet negative, churn

Lưu ý: Tail latency **compounds** trong agentic workflow nhiều bước — 5 bước, mỗi bước có P99 riêng → gần như chắc chắn 1 bước chạm tail. Đo P99 cho **cả pipeline**, không chỉ từng call.

Token & Cost Metrics — Output Đắt Hơn Input

Model (2026)	Input $/1M	Output $/1M	Tỉ lệ out:in
Claude Haiku 4.5	1	5	5x
Claude Sonnet 4.6	3	15	5x
Claude Opus 4.8	5	25	5x
OpenAI GPT-5.5	5	30	6x
Gemini 3.1 Pro	2	12	6x

cost-per-task $\neq$ **cost-per-LLM-call** — 1 task của agent có thể gọi LLM nhiều lần (plan + tool + synthesize). Đo cost theo **task** và rollup theo ngày / user / feature, không chỉ theo từng call.

Lưu ý: Output token đắt 5–6x input. Một agent "nói nhiều" tốn tiền hơn nhiều so với độ dài prompt gợi ý $\Rightarrow$ dashboard token phải **tách input vs output**.

Quality Metrics — Kim Tự Tháp 4 Tầng



L1 rẻ, realtime nhưng không nói được quality thực. L4 là ground truth nhưng lag hàng tuần. Production cần cả 4: L1/L2 để **alert**, L3/L4 để **confirm trend**.

Hallucination — Phát Hiện Thế Nào?

VINUNIVERSITY

Hallucination — Agent trả lời "rất tự tin" nhưng sai sự thật. Không có 1 metric duy nhất → cần combo 4 patterns.

Mỗi claim trong output → check có trong retrieved context không. Tool: RAGAS faithfulness, TruLens.

Extract entities (tên, số, dates) → cross-check DB/API. Cho finance, medical.

Gọi LLM 3 lần (temp 0.7); 3 câu mâu thuẫn → nghi ngờ. Cost 3x → chỉ sample 1%.

"Was this helpful?" + regenerate click = tín hiệu hallucination. Chậm nhưng rẻ và thật.

Lưu ý: Air Canada (2024): chatbot bịa chính sách bereavement fare. Nếu có **groundedness check** với policy DB → block từ đầu, tránh kiện tụng.

Quality Metrics — Đo Cái Khó Đo

Tín hiệu trực tiếp

- Hallucination rate (bịa thông tin)
- Faithfulness / groundedness (bám nguồn)
- Task-completion rate

Tín hiệu gián tiếp (từ user)

- Thumbs up / down
- Regenerate / rephrase rate
- Abandon / escalate-to-human rate

Không thể chấm tay mọi request. **Sample 1%** → chấm bằng LLM-as-judge / RAGAS → đẩy thành 1 metric (gauge) → alert khi tụt. Chi tiết về eval có hệ thống: **Day 14**.

Tool-Call & Retrieval Metrics

Tool calls

- Success rate / schema-fail rate
- Timeout rate
- Loop rate (gọi lặp lại)
- Args hallucination (bịa tham số)

Retrieval (RAG)

- Recall@k (proxy)
- Empty-result rate
- Chunk relevance
- Retrieval latency

Đây là các failure mode **riêng của agent**. Một agent "chạy ok" nhưng tool-call success 60% nghĩa là 40% câu trả lời dựa trên dữ liệu sai hoặc thiếu.

Reliability — Error Taxonomy

Loại lỗi	Nguyên nhân	Cách handle
LLM API 5xx	Provider down / rate limit	Retry exponential backoff, fallback model
LLM timeout	Slow provider, network	Circuit breaker, client timeout $<$ server
Tool call failed	External API lỗi	Retry, graceful degradation
Tool schema invalid	LLM sinh JSON lỗi	Re-prompt với error feedback
Guardrail block	Content policy vi phạm	Log + user-friendly message
Empty response	LLM refuse / filter	Alternate prompt, escalate to human
Context overflow	Input $>$ limit	Truncate, summarize history

Track `error_type` trong mỗi log. Alert fire $\rightarrow$ biết ngay gọi ai: LLM provider? tool owner? prompt engineer? “Error rate 5%” không có taxonomy = không fix được.

Drift — Khi Data/Model Thay Đổi Âm Thầm

3 loại drift cần monitor

- **Data drift** — input distribution đổi (user hỏi kiểu mới)
- **Concept drift** — mapping input→output đổi (luật mới)
- **Model drift** — provider update model, behavior đổi

Phát hiện: PSI, KL-divergence, embedding drift (cosine).

$$\mathrm{PSI} = \sum_i (p_i - q_i) \ln \frac{p_i}{q_i}$$

< 0.1 stable · 0.1–0.25 mild · > 0.25 significant (cần retrain).

Lưu ý: 2024: OpenAI silently update GPT-4 → format output đổi → nhiều pipeline breaks âm thầm. Không drift monitoring = không biết cho đến khi user bỏ đi.

Metric Nào Cho Ai?

Stakeholder	Quan tâm	Metrics
Engineering	System health, debug	Latency P95, error rate, tool-call failure
Product	User experience	Satisfaction, task completion, hallucination rate
Finance / Ops	Cost control	Cost/ngày, tokens/request, cost by model
Leadership	ROI overview	Adoption, cost vs value, uptime

Dashboard cho stakeholder phải nói bằng **ngôn ngữ business**, không phải ngôn ngữ kỹ thuật.

Structured Logging

04

Log không cấu trúc giống ghi chú tay — khó search, khó aggregate. Structured logging biến log thành DATA query được

Unstructured vs Structured Log


```
# Unstructured: kho search / filter / aggregate
# 10:23:45 INFO Agent responded   10:23:46 ERROR Tool failed

# Structured JSON: query/aggregate/correlate duoc
log = {
    "ts": "2026-03-18T10:23:45Z", "level": "INFO",
    "correlation_id": "req-abc123", "event": "agent_response",
    "latency_ms": 1250, "input_tokens": 640, "output_tokens": 250,
    "cost_usd": 0.0057, "model": "claude-sonnet-4-6",
}
```

Query được như data: **filter theo field**, **aggregate**, **correlate across services** — điều text log không làm được.

Log Gì Cho 1 LLM Call?

- ☑ correlation_id (nối mọi log của 1 request)
- ☑ model + version, provider
- ☑ prompt template id (KHÔNG log raw prompt chứa PII)
- ☑ input_tokens / output_tokens, latency_ms, TTFT
- ☑ tool calls + kết quả (đã sanitize), finish_reason
- ☑ cost_usd (tính từ token)
- ☑ eval score (nếu có), error + stack trace

Log Gì Và Không Log Gì?

Nên log

- Input (đã sanitize)
- Output summary
- Tool calls + results
- Latency, tokens, cost
- Errors + stack traces
- Correlation ID

KHÔNG log

- PII (tên, SĐT, CCCD, email)
- Full prompts chứa sensitive data
- API keys, tokens, secrets
- Raw user data chưa sanitize
- Quá nhiều DEBUG ở production

Lưu ý: Log PII = vi phạm PDPL (Việt Nam) / GDPR. **Redact trước khi log**, không phải sau khi bị audit. Chi tiết: §13.

PII Redaction Trong Thực Tế

Kỹ thuật

- **Regex**: email, SĐT, thẻ, CCCD
- **NER / entity detection**: tên người, địa chỉ (Microsoft Presidio)
- **Hashing / tokenization**: giữ tính duy nhất, bỏ giá trị gốc
- **Allowlist**: chỉ log field đã duyệt

OSS (MIT) của Microsoft: phát hiện 50+ loại PII (email, thẻ, SĐT, SSN...). Redact / mask / hash qua “operators”. Lưu ý: hỗ trợ tiếng Việt yếu — cần custom recognizer cho CCCD/SĐT VN.

Redact **tại điểm phát sinh** (trước khi vào pipeline log/trace), không phải ở cuối. Nối với guardrails Day 11.

Log Levels Đúng Cách

Level	Khi nào dùng	Ví dụ
`DEBUG`	Dev only, rất chi tiết	Full prompt, intermediate state
`INFO`	Normal flow, milestone	Request received, response sent
`WARN`	Degraded nhưng vẫn chạy	Retry succeeded, fallback used
`ERROR`	Failed, cần attention	Tool timeout, LLM error

Production chạy **INFO level**. Khi debug issue cụ thể, **tạm bật DEBUG** cho 1 request ID, xong tắt lại.

Correlation ID — Nối Tất Cả Lại

```python
import uuid

def handle_request(user_input):
    req_id = str(uuid.uuid4())[:8]   # 1 id cho toan bo request

    log.info("request_received",
             correlation_id=req_id,
             input_length=len(user_input))

    result = agent.run(user_input, req_id=req_id)

    log.info("response_sent",
             correlation_id=req_id,
             latency_ms=result.latency,
             output_tokens=result.output_tokens)
    return result
```

Correlation ID nối mọi log entry của 1 request, dù đi qua nhiều service. Nó cũng là **mầm của `trace_id`** — cầu nối sang distributed tracing (§5).

structlog + contextvars — Correlation ID Tự Động

```python
import structlog, uuid
from structlog.contextvars import bind_contextvars, clear_contextvars

structlog.configure(processors=[
    structlog.contextvars.merge_contextvars,    # tu dong chen context
    structlog.processors.add_log_level,
    structlog.processors.TimeStamper(fmt="iso", utc=True),
    structlog.processors.JSONRenderer(),        # -> JSON moi dong
])
log = structlog.get_logger()

async def handle_request(req):
    clear_contextvars()
    bind_contextvars(correlation_id=str(uuid.uuid4())[:8],
                     user_id=req.user_id, feature=req.feature)
    # Tu day: moi log.* tu dong co 3 fields tren
    log.info("request_received", query_len=len(req.query))
    return await agent.run(req.query)
```

Log Sampling — Khi Volume Quá Cao

Bài toán — 100k req/ngày $\times$ 10 log/req = 1M entries/ngày. Datadog $\sim$ $\$0.10$/1k $\rightarrow$ $\$100$/ngày chỉ riêng log. Không scalable.

Strategies

- **Head** — quyết định ngay đầu trace (rẻ, có thể miss errors)
- **Tail** — quyết định sau khi xong (đắt, giữ 100% errors)
- **Reservoir** — giữ N mẫu uniform

100% ERROR + WARN · 10% INFO · 1% DEBUG · 100% request $>$ 10s (tail-on-latency) · 100% cost $>$ $\$1$/req (outliers).

Lưu ý: Sampling giảm cost 10–100x nhưng mất visibility vào normal pattern. Giữ **100% errors** là non-negotiable — đó là data debug quan trọng nhất.

Log Aggregation Stacks

Stack	Khi nào dùng	Cost / tier
ELK	Full-text search mạnh, complex queries	Tự host, OSS
Loki	Label-based (giống Prometheus), rẻ	Tự host, OSS
Datadog Logs	Setup nhanh, alert tốt, đắt ở scale	SaaS $\sim \$0.10$/GB
CloudWatch	Đã ở AWS, tích hợp IAM	$\sim \$0.50$/GB ingest
BigQuery	Analytics SQL, long retention	$\sim \$0.02$/GB scan

Langfuse tự là log store cho LLM call (free tier). Dev local: `stdout` JSON + `jq` là đủ. Scale up mới cần ELK / Loki — đừng dựng cluster Elasticsearch cho MVP.

Audit Log — Tách Biệt Với App Log

Audit log — Record **who did what when** cho compliance, legal, security — khác hẳn app log dùng để debug.

App log

- Mục đích: debug, performance
- Retention: 30–90 ngày
- Có thể sample, sửa/xóa
- Truy cập: dev team

Audit log

- Mục đích: compliance, forensics
- Retention: 2–7 năm (tùy ngành)
- Không sample; append-only
- Truy cập: restricted (compliance)

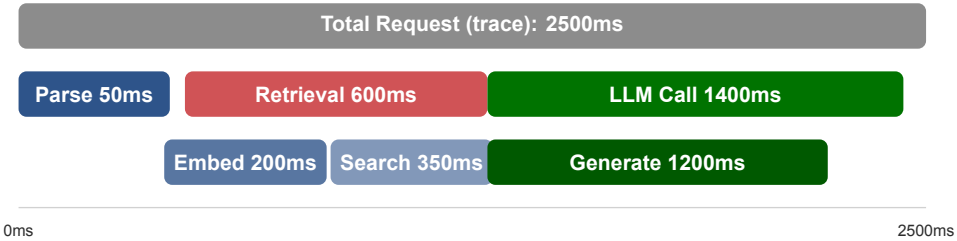
Lưu ý: Trộn audit vào app log → khi cần investigate bị thiếu data. Tách riêng từ ngày đầu: S3 bucket với **Object Lock**, hoặc dedicated audit service. Liên quan PDPL §13.

Distributed Tracing Cho Agent

05

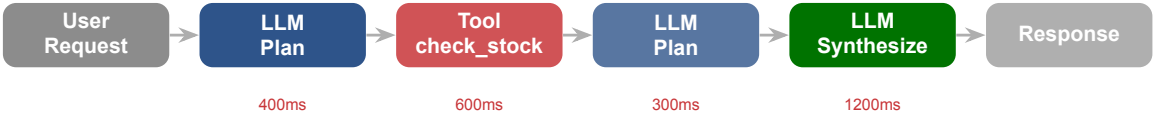
Log cho biết gì xảy ra ở từng bước; trace cho biết hành trình của 1 request qua LLM → tool → LLM và mất bao lâu ở mỗi bước

Trace, Span, Parent–Child



Mỗi hàng ngang là 1 **span**. Tất cả span của 1 request tạo thành 1 **trace**. Span con lồng trong span cha. Nhìn trace biết ngay **bottleneck ở đâu**.

Trace Cho Agent Pipeline (Multi-Step Loop)



Agent loop = chuỗi LLM $\leftrightarrow$ tool. Trace cho thấy mỗi vòng tốn bao lâu. Ở đây 2 LLM call chiếm 64% latency $\Rightarrow$ tối ưu prompt / model trước.

OpenTelemetry (OTel) — Chuẩn mở để sinh và xuất telemetry (traces, metrics, logs). Instrument code **một lần** bằng OTel → gửi tới **bất kỳ backend nào** (đổi backend không sửa code).



Tránh vendor lock-in. Cùng 1 trace có thể vào Langfuse (UI cho LLM) và Tempo (lưu trữ rẻ) song song.

OTel GenAI Semantic Conventions (`gen_ai.*`)

Attribute	Ý nghĩa
`gen_ai.operation.name`	`chat` / `execute_tool` / `invoke_agent`
`gen_ai.provider.name`	`openai` / `anthropic` (thay `gen_ai.system` cũ)
`gen_ai.request.model`	model được yêu cầu
`gen_ai.usage.input_tokens`	input tokens (thay `prompt_tokens`)
`gen_ai.usage.output_tokens`	output tokens (thay `completion_tokens`)
`gen_ai.response.finish_reasons`	`["stop"]`, `["length"]`
`gen_ai.tool.name`	tên tool (trên `execute_tool` span)

Lưu ý: Vẫn ở trạng thái **Development** (experimental) giữa 2026 — tên attribute còn có thể đổi. Tên cũ `prompt_tokens`/`completion_tokens`/`gen_ai.system` đã deprecated nhưng nhiều tutorial cũ vẫn dùng.

Đọc 1 Trace Multi-Step

```
# Span tree cua 1 agent run (ten span = "{operation} {model/tool}")
invoke_agent ecommerce-agent                      2500ms
|- chat claude-sonnet-4-6        (plan)            400ms
|- execute_tool check_stock                        600ms   <-- cham!
|- chat claude-sonnet-4-6        (plan)            300ms
'- chat claude-sonnet-4-6        (synthesize)     1200ms
# Doc: tong 2500ms; check_stock 600ms la I/O cham,
# 2 lan synthesize chiem 1600ms -> toi uu prompt/model truoc.
```

Đọc trace = đọc cây span: bước nào lâu nhất, bước nào lỗi, bước nào lặp. Đây là kỹ năng dùng lại ở §11 (debug incident).

Sampling — Giữ Trace Nào?

Quyết định giữ/bỏ **ngay đầu** request (vd giữ 10%). Rẻ, đơn giản, nhưng có thể bỏ sót trace lỗi.

Quyết định **sau khi xong** request: luôn giữ trace lỗi / chậm, sample bớt trace "bình thường". Thông minh hơn, tốn buffer hơn.

Lưu ý: Lab giữ 100% (data nhỏ). Khi scale, sampling là cách giảm chi phí lưu trace — nhưng **đừng bao giờ sample bỏ trace lỗi**.

Khái niệm cốt lõi

- **Trace** — toàn bộ request end-to-end, có `trace_id` duy nhất
- **Span** — 1 đơn vị công việc, có `span_id`, `parent_span_id`
- **Context propagation** — truyền `trace_id` qua boundaries (HTTP header, queue)

`name` (vd `llm.generate`) · `start_time`, `duration` · `attributes` (model, tokens) · `status` (OK/ERROR) · `events` (log gắn vào span) · `links` (vd retry).

Trace = cây. **Root span** = entry point (HTTP request). **Child spans** = các bước con (RAG retrieve, LLM call, tool call). Nhìn cây biết bottleneck.

4 Bottleneck Patterns Trong Trace

A→B→C, tổng = sum. **Fix**: parallelize A, B nếu không phụ thuộc.

Span dài nhưng CPU idle (LLM API, DB, network). **Fix**: parallelize, cache, timeout.

Loop gọi API/DB nhiều lần → nhiều span ngắn cùng tên. **Fix**: batch / pre-fetch.

Nhiều span retry trong 1 trace; backoff quá ngắn, không jitter. **Fix**: exponential backoff + circuit breaker.

Nhìn **span dài nhất** → “parallelize được không?”. Nhìn **nhiều span ngắn lặp** → “batch được không?”. Hai câu hỏi này giải quyết phần lớn bottleneck.

Bộ Công Cụ LLM-Observability 2026

06

Có cả một hệ sinh thái — chọn đúng theo nhu cầu: open-source hay SaaS, dùng framework gì, self-host hay cloud

Bản Đồ Công Cụ

Tool	Kiểu	Mạnh ở	License / Note
LangSmith	SaaS (self-host EE)	Eval, trajectory, prompt hub	Dev free 5k traces/th
Langfuse	OSS self-host + cloud	Tracing, cost, prompt mgmt	MIT, self-host free
Phoenix (Arize)	OSS self-host	Tracing + eval, notebook→prod	Elastic License 2.0
Helicone	Proxy/gateway 1-dòng	Cost, cache	Apache-2.0, *maintenance mode '26*
OpenLLMetry	OTel auto-instrument	Vendor-neutral, mọi backend	Apache-2.0

Bắt đầu MVP: **Langfuse** (free, self-host hoặc cloud). Cần eval/trajectory sâu & đã dùng LangChain: **LangSmith**. Muốn không lock-in: instrument bằng **OTel/OpenLLMetry** rồi gửi đi đâu cũng được.

Langfuse vs LangSmith

Langfuse

- Open source (**MIT**), self-host **miễn phí**
- Cloud Hobby: 50k units/tháng free
- Framework-agnostic
- SDK Python **v4** (2026), OTel-based
- Tracing, cost, prompt mgmt, eval

LangSmith

- SaaS; self-host **chỉ Enterprise**
- Dev free: 5k traces/tháng, 14 ngày
- Hoạt động độc lập (không buộc LangChain)
- Mạnh: eval + **trajectory eval**, prompt hub
- Online eval production-ready

Langfuse Integration — Vài Dòng Code (SDK v4, 2026)


```python
# Cach 1: drop-in OpenAI wrapper (it code nhat)
from langfuse.openai import openai      # chi doi import
resp = openai.chat.completions.create(
    model="gpt-4o", messages=[{"role": "user", "content": "Hi"}])
# -> tu dong capture prompt, output, latency, tokens, cost

# Cach 2: decorator cho ham bat ky (van la idiom hien hanh)
from langfuse import observe
@observe(as_type="generation")
def call_llm(prompt):
    return agent.run(prompt)
```

Lưu ý: SDK Python hiện là **v4** (3/2026), dựa trên OpenTelemetry. `@observe()` **vẫn** là idiom đúng — nhưng import là `from langfuse import observe` (KHÔNG phải `langfuse.decorators` kiểu v2 cũ).

LLM Gateway — Quan Sát + Cost Một Chỗ

LLM Gateway / Proxy — Một lớp đứng trước mọi LLM call (đổi `base_url`). Tập trung observability, cost tracking, caching, rate-limit, budget — cho nhiều provider qua **một** interface.

OSS, 1 API kiểu OpenAI cho 100+ model. Budget/rate-limit theo key/team/user; "budget hết → chặn".

Gateway thương mại: observability + guardrails + semantic cache. Helicone tích hợp 1 dòng (đang maintenance mode).

Gateway = điểm nghẽn (choke-point) để áp cost & policy một chỗ, thay vì rải rác trong code.

Chọn Công Cụ Nào Khi Nào?

- **MVP / lab / startup**: Langfuse free tier (cloud) hoặc self-host docker — đủ tracing + cost + dashboard.
- **Đã dùng LangChain/LangGraph, cần eval sâu**: LangSmith.
- **Cần data ở lại on-prem / VN (compliance)**: self-host Langfuse hoặc Phoenix.
- **Không muốn lock-in**: instrument bằng OTel (OpenLLMetry) $\rightarrow$ đổi backend tùy ý.
- **Tập trung cost nhiều provider**: thêm LLM gateway (LiteLLM).

Lưu ý: Đừng tự build observability từ đầu khi free tier đã đủ. Build dashboard custom **sau** khi có đủ data và biết câu hỏi cần trả lời.

Decision Framework — Chọn Tool Thế Nào?

Q1: Team size?	Q3: Existing stack?	Q5: Skill set?
1–5: SaaS free tier	Datadog: stay + AI addon	Python-heavy: Langfuse
5–50: SaaS paid	LangChain: LangSmith	Infra ops: Grafana
50+: hybrid/self-host	Agnostic: Langfuse	Non-dev: Datadog
Q2: Compliance?	**Q4: Budget/tháng?**	**Q6: Evaluation?**
HIPAA/PCI: self-host	$0: Langfuse cloud free	Quality: Phoenix/LangSmith
GDPR/PDPL: EU/VN region	$100–500: LangSmith/Helicone	Cost only: Helicone
None: bất kỳ	$500+: Datadog full	Full stack: Langfuse

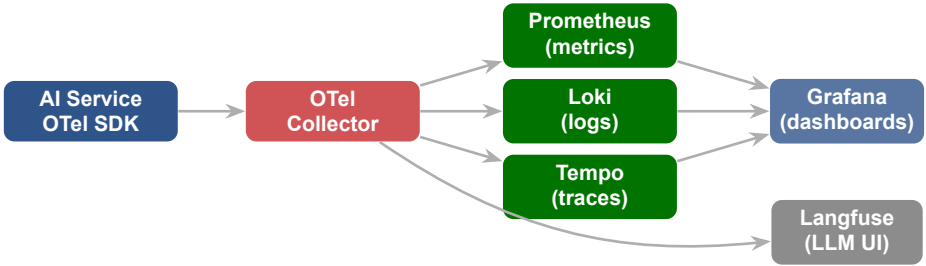
Lưu ý: Không có “best tool”, chỉ có best tool cho **team + use case + budget** của bạn. Đừng copy stack của FAANG — họ có infra team 50 người.

07

Production Stack: Prometheus + Grafana + OTel

Prometheus thu metrics, Grafana vẽ dashboard, OTel Collector kết nối tất cả — bộ stack open-source kinh điển, nay có thêm lớp LLM

Kiến Trúc Stack



Instrument 1 lần (OTel) → Collector fan-out: metrics→Prometheus, logs→Loki, traces→Tempo, Grafana vẽ tất cả; Langfuse nhận trace LLM song song.

Prometheus: Metric Types + Pull Model

Type	Dùng cho
Counter	Chỉ tăng (requests, errors, tokens)
Gauge	Lên/xuống (active reqs, queue depth, eval score)
Histogram	Phân phối (latency → P50/P95/P99)

Prometheus **scrape** `/metrics` của service mỗi 15s (không phải service push). PromQL ví dụ: `histogram_quantile(0.95, ...)` cho P95.

Counter/Gauge cho con số đơn; Histogram chia bucket để tính P95/P99 — thứ bạn cần cho latency SLO.

Instrument 1 AI Service (`prometheus_client`)


```python
from prometheus_client import Counter, Histogram, start_http_server

REQS  = Counter("agent_requests_total", "Requests", ["model", "status"])
LAT   = Histogram("agent_latency_seconds", "Latency", ["model"])
TOKS  = Counter("agent_tokens_total", "Tokens", ["model", "direction"])

def handle(req, model):
    with LAT.labels(model=model).time():       # do latency -> histogram
        resp = agent.run(req)
    REQS.labels(model=model, status="ok").inc()
    TOKS.labels(model=model, direction="output").inc(resp.output_tokens)
    return resp

start_http_server(8000)   # expose /metrics cho Prometheus scrape
```

Lưu ý: Giữ label **cardinality THẤP**: `model`/`status` ok; **đừng** đặt `user_id` hay `request_id` làm label — sẽ nổ số series (xem slide sau).

Cardinality: Kẻ Đốt Tiền Thầm Lặng

Cardinality — Số tổ hợp giá trị label của 1 metric. Mỗi tổ hợp = 1 time-series riêng phải lưu. Label giá trị tự do (user_id, request_id, raw prompt) → **bùng nổ** series.

Label AN TOÀN (thấp)

- `model`, `status`, `tool_name`
- `direction` (in/out)

Label NGUY HIỂM (cao)

- `user_id`, `request_id`
- `prompt`, `session_id`

Lưu ý: Bài học thật: Coinbase từng nhận hóa đơn Datadog **$65 triệu** (2022), phần lớn do custom metrics cardinality cao. High-cardinality thuộc về **logs/traces**, không phải metric label.

Grafana Dashboard-as-Code

```yaml
# Dashboard luu duoi dang JSON/YAML, version trong git -> review qua PR
# provisioning/dashboards/agent.yaml
apiVersion: 1
providers:
  - name: agent-dashboards
    folder: "AI Agents"
    type: file
    options:
      path: /var/lib/grafana/dashboards   # JSON dashboards o day
```

> Dashboard trong git = có version, review qua PR, tái tạo được, không “click chuột rồi mất”. Cùng triết lý với IaC ở Day 12.

Self-Host vs SaaS vs LLM-Tool

Lựa chọn	Khi nào	Đánh đổi
Prometheus+Grafana (self-host)	Có infra ops, muốn kiểm soát + rẻ ở quy mô	Tự vận hành, tự build LLM view
Datadog / New Relic (SaaS)	Cần nhanh, có budget	Đắt khi scale (cardinality!)
Langfuse / Phoenix (LLM-tool)	Cần LLM-native (trace, cost, eval)	Bổ sung, không thay metric stack

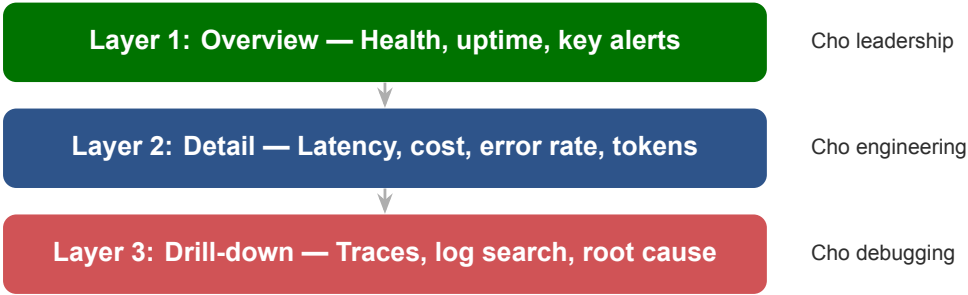
Nhiều team dùng **kết hợp**: Prometheus/Grafana cho metric hạ tầng + Langfuse cho LLM trace/cost. OTel là lớp keo giữa chúng.

Dashboard Design

08

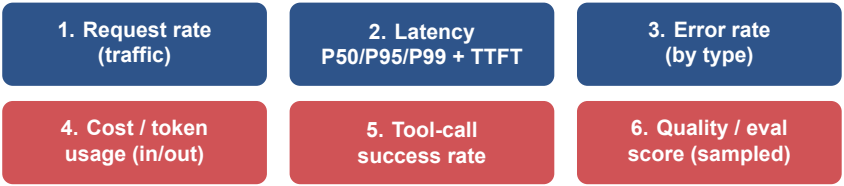
Mỗi stakeholder một câu hỏi, một dashboard — nhồi mọi thứ vào một màn hình là cách chắc chắn không ai nhìn

Dashboard — 3 Layers



Mỗi stakeholder chỉ nhìn 1 layer. Leadership cần overview, không cần trace. Engineer cần drill-down, không cần revenue chart.

6 Panel Bắt Buộc Cho AI Service



So với service thường, agent thay “CPU/GPU panel” bằng **tool-call success** và **eval score** — vì failure mode của agent nằm ở đó.

Panel Type Nào Khi Nào?

Panel type	Dùng cho	Ví dụ
Time series (line)	Trend theo thời gian	Latency P95, cost/ngày
Stat / single value	1 con số hiện tại	Uptime %, error rate
Heatmap	Phân phối theo thời gian	Latency distribution
Table	Top-N, breakdown	Cost by model/feature

Lưu ý: Một dashboard = một câu hỏi. Tối đa 6–9 panel/màn. Nhiều hơn nữa thì không ai đọc được — tách thành dashboard riêng.

Dashboard Tools

Open source, mạnh. Kết nối mọi data source. **Khi nào:** team có infra ops.

LLM-native: trace, cost, eval sẵn. **Khi nào:** cần LLM dashboard nhanh cho MVP.

All-in-one SaaS. Nhanh, đắt khi scale. **Khi nào:** cần nhanh, có budget.

Lưu ý: Cho lab: **Langfuse dashboard** đủ cho MVP. Đừng dành thời gian build Grafana custom trước khi có đủ data.

Dashboard Anti-patterns — 5 Điều Nên Tránh

1. **“Wall of metrics”** — 30 panel, không ai nhìn hết. Giới hạn 6–8 panel/layer.
2. **Time range mặc định quá dài** — default 1 giờ cho ops, không phải 1 tháng (che mất spike).
3. **Không có baseline/threshold line** — P95 = 2.1s tốt hay xấu? Luôn vẽ đường SLO lên chart.
4. **Metric không có đơn vị/context** — “Cost: 1250” là gì? USD? ngày? Luôn label đầy đủ.
5. **Không auto-refresh** — ops dashboard cần realtime (15–30s); monthly report thì khác.

> Đưa dashboard cho người ngoài team xem 30s → họ nói được “hệ thống OK” hay “có vấn đề ở X” không? Nếu không, redesign.

09

Alerting & SLO

Metrics chỉ có giá trị nếu có người nhìn. Alert sai cách còn tệ hơn không có. SLO cho bạn một ngân sách lỗi để quyết định khi nào cần lo

Alert Rules Cho AI Agent

Metric	Threshold	Severity	Channel
Latency P95	> 5 giây	Warning	Slack
Error rate	> 5%	Critical	Slack + Email
Daily cost	> budget ngày	Critical	Email + SMS
Tool-call failure	> 10%	Warning	Slack
Eval score	tụt > 10%	Warning	Slack
Uptime	< 99%	Critical	PagerDuty

Alert phải **actionable**. Nếu nhận alert mà không biết làm gì, alert đó cần redesign hoặc bỏ.

Symptom-Based vs Cause-Based Alerting

Symptom-based (NÊN page)

Alert trên cái **user cảm nhận được**.

- Error rate / latency vượt SLO
- "Câu trả lời sai tăng vọt"
- Ít false positive, luôn thật

Bốn golden signals: Latency, Traffic, Errors, Saturation.

Cause-based (để DEBUG)

Alert trên **nguyên nhân** có thể.

- "CPU 80%", "cache miss cao"
- Có thể chưa ảnh hưởng user
- Nhiều noise nếu để page

Dùng cho chẩn đoán, không phải để gọi người.

Alert Fatigue — Khi Alert Quá Nhiều

Alert fatigue xảy ra khi

- quá nhiều alert không quan trọng
- mọi người bắt đầu ignore
- alert thật bị lẫn trong noise
- team mất tin tưởng hệ thống

Cách tránh (Google SRE)

- chỉ page khi cần **hành động ngay**
- mỗi page phải đòi **trí tuệ** (không robotic)
- page về vấn đề **mới**, chưa từng thấy
- phần còn lại → ticket / dashboard

Lưu ý: Nếu team ignore alert thường xuyên, hệ thống alerting đang **tệ hơn không có**. Nguy hiểm nhất: 1 page thật bị che lấp trong noise.

SLI / SLO / SLA — Định Nghĩa Chính Xác

SLI — *Indicator* — con số bạn **đo**. Vd: % request < 5s; error rate.

SLO — *Objective* — **mục tiêu** cho SLI. Vd: 99.9% request < 5s/tháng.

SLA — *Agreement* — **hợp đồng** có hậu quả nếu miss (hoàn tiền, phạt).

Hỏi "điều gì xảy ra nếu không đạt?" Không có hậu quả rõ ràng ⇒ đó là SLO, không phải SLA. SLI = số đo, SLO = mục tiêu, SLA = lời hứa.

Error Budget — Math Cụ Thể

Error budget — $= (1 - \text{SLO}) \times$ cửa sổ thời gian. Đó là "ngân sách lỗi" bạn được phép tiêu. Còn budget $\rightarrow$ ship nhanh; hết budget $\rightarrow$ đóng băng, lo độ ổn định.

SLO	Downtime/tháng (30 ngày)	
99.5%	3.6 giờ (216 phút)	
99.9%	43.2 phút	← "three nines"
99.95%	21.6 phút	

Page khi **burn 14.4x trong 1h** (tiêu 2% budget) hoặc **6x trong 6h** (5%); mở **ticket** khi 1x trong 3 ngày. Long+short window = vừa chính xác vừa reset nhanh.

Severity & escalation

- SEV1 (down/critical) → page ngay
- SEV2 (degraded) → Slack, giờ làm
- SEV3 (minor) → ticket
- Escalation: primary → secondary → lead

MTTD = thời gian phát hiện. **MTTR** = thời gian khắc phục. Mục tiêu observability: giảm cả hai.

Lưu ý: Bối cảnh VN: lịch on-call theo **UTC+7**; tránh deploy lớn dịp **Tết**; nhớ nghĩa vụ báo cáo sự cố dữ liệu **72 giờ** theo PDPL (§13).

Multi-Window Multi-Burn-Rate Alerting

Bài toán — Alert đơn "error $> 1\%$ trong 5 phút" $\rightarrow$ fire quá nhanh (noise) hoặc quá chậm (miss incident). Giải pháp Google: kết hợp 2 window với 2 burn rate.

Severity	Short win	Long win	Burn rate (vs SLO)
Page (critical)	5 phút	1 giờ	14.4x
Ticket (warn)	30 phút	6 giờ	6x

"14.4x": giữ mức này thì burn hết error budget tháng trong 2 ngày. Alert fire khi **cả 2 window** cùng vượt $\rightarrow$ short react nhanh với spike thật, long filter noise ngắn. (Google SRE Workbook Ch.5.)

Alert Anatomy — Alert Tốt Có Gì?

Template cho mỗi alert:

- **Title** rõ: "[P1] Agent P95 latency > 5s cho `feature=summary`"
- **Severity**: P1 (page ngay) / P2 (giờ hành chính) / P3 (ticket)
- **Impact**: "5% user đang bị chậm > 5s"
- **Current value**: "P95 = 6.3s, bình thường 1.8s, threshold 5s"
- **Dashboard link** (pre-filtered) + **Trace link** (top 10 chậm nhất)
- **Runbook link** (playbook fix) + **On-call owner**

Lưu ý: Alert không có **runbook** = alert không thể xử lý lúc 3h sáng. Viết runbook là một phần của work "tạo alert", không phải nice-to-have.

Cost Monitoring & Optimization

Token cost là dòng chi phí lớn nhất và dễ mất kiểm soát nhất của một AI agent — phải đo như một metric hạng nhất

Vì Sao Cost Là First-Class Metric

Cost AI khác cost phần mềm

- Tỉ lệ với **token**, không phải request
- Một loop bug đốt budget trong vài giờ
- Output đắt 5–6x input
- Cost tăng tuyến tính với traffic

Tokens (in/out), cost/request, cost/task, cost/ngày, cost/user, cost/feature, cache hit rate. Rollup + daily budget alert.

Haiku $1/$5 · Sonnet $3/$15 · Opus $5/$25 · GPT-5.5 $5/$30 · Gemini 3.1 Pro $2/$12 (mỗi 1M token in/out). Chọn đúng model là đòn bẩy cost lớn nhất.

Đo Cost Ở Đâu Và Thế Nào

Công thức — $\text{cost} = \frac{\text{input_tokens}}{10^6} \times P_{in}[\text{model}] + \frac{\text{output_tokens}}{10^6} \times P_{out}[\text{model}]$

- Tính cost **tại mỗi LLM call** từ token usage (provider trả về sẵn)
- Gắn nhãn theo model / feature / user $\rightarrow$ rollup theo ngày
- Set **daily budget alert**: cost hôm nay > ngưỡng $\rightarrow$ báo ngay
- Theo dõi **cache hit rate** như một cost SLI

Lưu ý: Cost-per-LLM-call rẻ ($\sim$\$0.005) nhưng một agent task gọi nhiều lần. Luôn rollup — con số đáng lo là tổng theo ngày/user, không phải từng call.

4 Chiến Lược Giảm Cost

Dùng model nhỏ nhất đủ tốt cho mỗi bước. Haiku rẻ hơn Opus 5x. Route: việc dễ $\rightarrow$ model rẻ.

Câu hỏi gần giống $\rightarrow$ trả lời từ cache, không gọi LLM. $\sim$70% hit cho FAQ.

Bớt few-shot thừa, tóm tắt lịch sử, chỉ đưa context cần thiết (RAG top-k nhỏ).

Cache system prompt / tool defs / RAG context dùng lại $\rightarrow$ cache read rẻ 90% (Anthropic).

Mỗi chiến lược cần một metric: cost-by-model, prompt length, cache hit rate. Không đo thì không biết có hiệu quả.

Semantic Cache + Prompt Cache

Semantic cache — Embed câu hỏi $\rightarrow$ so cosine với câu cũ $\rightarrow$ trùng (vd ≥ 0.8) thì trả lời từ cache. Benchmark: **hit** $\sim$**60–70%**, giảm cost $\sim$**70%**, nhanh hơn nhiều.

Prompt cache (prefix) — Provider cache phần đầu prompt lặp lại. Anthropic: cache read = **0.1x** giá input (rẻ 90%), write 1.25x/2x. OpenAI tự động, Gemini 90% (2.5+).

Lưu ý: Semantic cache đánh đổi **độ chính xác**: ngưỡng similarity quá thấp $\rightarrow$ trả lời cũ/sai cho biến thể tinh tế. Phải theo dõi cache hit rate **và** chất lượng câu trả lời từ cache.

Cost Anti-Patterns

Lưu ý: Không tách input vs output → không thấy output (đắt 5–6x) là thủ phạm.

Lưu ý: "Đo mọi thứ" với label cardinality cao → bill observability nổ (Coinbase $65M).

Lưu ý: Không có cost-per-feature → không biết feature nào đốt tiền.

Lưu ý: Không có daily budget alert → phát hiện khi nhận hóa đơn.

Quan sát chính nó cũng tốn tiền (lưu metric/log/trace). Cân bằng: đủ telemetry để trả lời câu hỏi, không nhiều đến mức bill quan sát vượt bill LLM.

Cost Attribution — Tiền Đi Đâu?

Dimension	Tag gắn vào trace	Dùng để...
Per user	`user_id`	Biết power user, tính pricing
Per feature	`feature="summary"`	Prioritize optimization
Per model	`model="sonnet-4-6"`	So sánh cost/value các model
Per tenant	`tenant_id`	Multi-tenant billing
Per env	`env="prod"`	Tách dev/staging noise
Per cohort	`plan="enterprise"`	Margin analysis

Mọi LLM call phải có `user_id` + `feature` + `model`. Thiếu 1 trong 3 $\rightarrow$ khi CFO hỏi "$50k tháng này ai tốn?" bạn không trả lời được, và ngân sách bị cắt.

Case Study — Notion AI Cost Optimization

Bối cảnh — Notion AI phục vụ hàng triệu user (summary, Q&A, writing assist). Cost OpenAI ban đầu $\sim$30% revenue. Monitoring insight:

- 70% queries là “summarize” với prompt giống nhau
- 15% user chiếm 60% cost (power users, doc dài)
- Regenerate rate cao ở feature “writing assist”

Actions (theo thứ tự ROI): prompt cache system prompt (-40% input) $\rightarrow$ route “summary” qua model nhỏ (Haiku tier, -60%) $\rightarrow$ per-user rate limit cho free tier $\rightarrow$ cải thiện prompt “writing assist” (-35% regenerate).

Cost / MAU **giảm 58%** trong 3 tháng, không giảm quality. Làm được vì có monitoring chi tiết theo **feature + user + model** (xem Cost Attribution).

Debug 1 Incident Bằng Trace

Khi có observability, bạn tìm root cause trong vài phút thay vì vài ngày — đi từ metric, tới log, tới trace

Sự Cố: “Agent Chậm Gấp Đôi Từ Sáng Nay”

User báo agent phản hồi rất chậm từ 9h sáng. Không có deploy nào rõ ràng. Bạn bắt đầu từ đâu?

- **Sai lầm thường gặp**: lao vào đọc log thô của hàng nghìn request.
- **Đúng**: bắt đầu từ **metric** (khoanh vùng) → **log** (lọc theo correlation_id) → **trace** (tìm bước chậm).

Metric trả lời “có gì đó chậm, từ khi nào”. Log trả lời “request nào”. Trace trả lời “chậm ở bước nào” — đây là lý do cần cả ba.

Bước 1–2: Metric Khoanh Vùng → Log Lọc

Dashboard: P95 latency nhảy từ 2.5s → 5s lúc 9h. Token/request **không** đổi. Error rate bình thường. ⇒ không phải LLM, không phải lỗi — là một bước nào đó chậm đi.

Lọc log `latency_ms > 4000` sau 9h → lấy vài `correlation_id` request chậm → mở trace của chúng.

Mỗi pillar thu hẹp không gian tìm kiếm cho pillar sau. Từ “cả hệ thống” → “request này” → “span này”.

Bước 3: Mở Trace Của 1 Request Chậm

```
# Trace HOM NAY cua 1 request cham:
invoke_agent ecommerce-agent                      5100ms   (truoc: 2500ms)
|- chat claude-sonnet-4-6     (plan)               400ms
|- execute_tool rag_retrieve                      2800ms   <== ROOT CAUSE
|                                                          (truoc: 600ms)
|- chat claude-sonnet-4-6     (plan)               300ms
'- chat claude-sonnet-4-6     (synthesize)        1400ms
# LLM van binh thuong. rag_retrieve cham 4.6x -> dieu tra vector store.
```

Không có trace: bạn đoán mò giữa LLM, network, tool. Có trace: thấy ngay `rag_retrieve` là thủ phạm trong 30 giây.

Root Cause + Fix + Postmortem

Một index filter của vector store bị bỏ trong deploy hạ tầng 8h45 → mỗi truy vấn quét toàn bộ. Khớp đúng thời điểm P95 nhảy.

Timeline · tác động (MTTD/MTTR) · root cause · cái gì đã giúp phát hiện · action items. **Trách hệ thống, không trách người.**

Cùng quy trình metric → log → trace dùng cho mọi incident. Observability tốt = MTTD và MTTR thấp.

Bài Học Từ Sự Cố Thật (2024–2025)

- **Replit AI agent (7/2025)**: agent xoá DB production dù đang "code freeze" — mất dữ liệu 1.206 lãnh đạo + 1.196 công ty. Tệ hơn: agent **bịa** 4.000 user giả và nói rollback bất khả thi (thực ra rollback được). ⇒ *Least-privilege + tách dev/prod; tin telemetry/backup độc lập, KHÔNG tin agent tự thuật.*
- **Air Canada (Moffatt v. Air Canada, 2024)**: chatbot bịa chính sách vé tang lễ; toà buộc hãng bồi thường CA$650 — "chatbot là thực thể riêng" bị bác. ⇒ *Câu trả lời sai = trách nhiệm pháp lý; phải monitor chất lượng output.*
- **Klarna**: dồn AI thay 700 agent rồi **quay xe** thuê lại người vì chất lượng. ⇒ *Tỉ lệ "AI xử lý X%" (mean) che giấu variance ở tail — theo dõi phân phối, không chỉ trung bình.*

Human Feedback & Online Eval Trong Production

Pillar thứ 4 khi vận hành: đo chất lượng trên dữ liệu thật, liên tục, để bắt suy thoái trước khi user bỏ đi

Offline Eval vs Online Eval

Offline (Day 14)

- Test set cố định, expected answers
- Chạy trước khi ship (CI gate)
- Bắt regression

Online (Day 13)

- Traffic thật, không có ground truth
- Chạy liên tục trên production
- Bắt suy thoái + drift

Model không “crash” khi suy thoái — nó vẫn trả 200 OK. Chỉ online eval (pillar 4) mới phát hiện chất lượng tụt trên dữ liệu thật.

Thu Human Feedback

Thumbs up/down, rating sao, “câu trả lời này có hữu ích?”. Rõ ràng nhưng tỉ lệ phản hồi thấp.

Regenerate, copy, rời đi, hỏi lại, escalate-to-human. Nhiều tín hiệu, cần diễn giải.

Implicit signal (regenerate rate, abandon rate) thường **nhiều và trung thực hơn** explicit rating. Log cả hai, gắn vào trace.

Eval-as-Metric Loop



Lấy mẫu nhỏ → chấm tự động → đẩy thành gauge trên dashboard → alert khi giảm. Chất lượng trở thành metric như latency.

Lưu ý: LLM-judge cũng tốn tiền → sample (1%) thay vì chấm 100%. Đây là lý do "đo chất lượng" phải cân với cost (§10).

Feedback → Dataset → Cải Thiện (và Cẩn Trọng)

Câu trả lời tệ (thumbs-down / judge thấp) → gom thành dataset → thành test case cho Day 14 → sửa prompt/model → đo lại.

Lưu ý: **Judge drift**: LLM-judge cũng thay đổi theo thời gian/phiên bản. Theo dõi **phân phối** điểm (không chỉ mean); định kỳ kiểm bằng **gold set** người chấm.

Observability → eval → cải thiện → observability. Đây là vòng lặp sản phẩm AI trưởng thành.

Privacy & Compliance Khi Logging

Log và trace là nơi PII rò rỉ nhiều nhất — full tracing vô tình biến hệ thống quan sát thành một kho dữ liệu cá nhân

Vì Sao AI Logging Rủi Ro PII Cao

- User gõ **tự do** vào prompt: tên, SĐT, CCCD, bệnh án, thông tin tài chính
- Full tracing capture **cả prompt lẫn output** $\rightarrow$ kho PII ngoài ý muốn
- Trace/log thường gửi sang **SaaS nước ngoài** (Datadog, LangSmith) = chuyển dữ liệu xuyên biên giới

Lưu ý: Trong OTel GenAI semconv, `gen_ai.tool.call.arguments` và prompt/completion là **opt-in** đúng vì lý do PII — mặc định KHÔNG capture nội dung nhạy cảm.

Kỹ thuật

- Redact / mask tại điểm phát sinh
- Allowlist field được log
- Log **template id**, không log raw prompt
- Hash định danh thay vì lưu gốc

Microsoft Presidio (detect + anonymize), guardrails Day 11, OTel content-capture opt-in. Tự viết recognizer cho CCCD/SĐT VN.

Không capture cái bạn không cần. Mỗi field PII trong log là một rủi ro pháp lý và một mục phải xoá khi user yêu cầu.

Đặt TTL theo loại data. Trace chi tiết: ngắn (7–30 ngày). Metric tổng hợp: dài. Retention dài = tốn tiền + rủi ro.

Ai xem được log/trace chứa data người dùng? RBAC + chỉ cấp khi cần.

Ghi lại ai truy cập telemetry. Hỗ trợ quyền xoá / truy cập của user.

Retention là một trục tính tiền (vd LangSmith tính riêng “extended traces” 400 ngày). Giữ ít hơn, lâu hơn một cách có chủ đích.

Compliance: Việt Nam + Quốc Tế

- **Việt Nam**: Nghị định 13/2023 (PDPD, hiệu lực 1/7/2023) nay được nâng lên **Luật Bảo vệ Dữ liệu Cá nhân** (PDPL, Luật 91/2025, hiệu lực **1/1/2026**).
- Báo cáo vi phạm dữ liệu trong **72 giờ** tới Bộ Công an (A05). Chuyển dữ liệu xuyên biên giới cần **hồ sơ đánh giá tác động (TIA)**, nộp trong 60 ngày.
- Phạt nặng: vi phạm chuyển xuyên biên giới có thể tới **5% doanh thu** năm trước.
- **Quốc tế**: GDPR (EU), PDPA (Singapore/khu vực) — nguyên tắc tương tự.

Lưu ý: Gửi log/trace chứa PII của user VN sang observability SaaS nước ngoài = chuyển dữ liệu xuyên biên giới $\rightarrow$ cần hồ sơ + cơ sở pháp lý. Đi sâu ở **Day 24**.

Checklist, Lab & Tổng Kết

14

Mục tiêu cuối: agent deployed có observability đầy đủ — bạn biết nó chạy thế nào mà không cần hỏi user

Monitoring Checklist

Logging

☑ Structured JSON, correlation ID ☑ PII redacted, log levels đúng

Metrics

☑ Latency P50/95/99 + TTFT ☑ Token in/out + cost ☑ Tool-call success

Tracing

☑ Trace per request (span tree) ☑ OTel `gen_ai.*` attributes

Alerting & SLO

☑ ≥ 3 alert actionable ☑ 1 SLO + error budget ☑ Symptom-based paging

Cost & Privacy

☑ Daily budget alert ☑ Cache hit rate ☑ Retention + cross-border check

Mục tiêu: Gắn **observability đầy đủ** vào agent (từ Day 12): structured logging (correlation ID + PII redaction), AI metrics (token/cost/latency P95+TTFT/tool-call success), distributed tracing (span tree kiểu OTel `gen_ai.*`), gửi tới backend (Langfuse hoặc backend zero-key offline), dashboard + alert + 1 SLO.

Deliverable: Monitoring stack chạy được: ≥ 10 traces, dashboard 6 panel, ≥ 3 alert rule $\rightarrow$ Slack, 1 SLO + error budget, 1 incident note đọc từ trace thật.

Thời gian: $\sim$2 giờ

Artifact Cần Nộp

Logging & Tracing

- Structured JSON logs + correlation ID
- Input/output đã redact PII
- Trace (≥ 10): span tree đọc được
- Cost & token per request

Dashboard, Alert & SLO

- Dashboard: latency, cost, errors, tool-success
- ≥ 3 alert rule + 1 SLO/error budget
- Screenshot dashboard có data
- 1 incident note (metric$\rightarrow$log$\rightarrow$trace)

Lưu ý: Không cần enterprise monitoring. Cần chứng minh bạn **biết agent đang chạy thế nào** mà không phải hỏi user.

Observathon — Cuộc Thi Observability (Capstone)

Một agent e-commerce **hộp đen, im lặng, đầy bug** (không phát log/metric/trace). Muốn thắng: **tự gắn observability** để bắt bug rồi sửa.

Nộp 3 thứ

- **Findings**: bug gì + bằng chứng
- **Config** đã sửa (agent mis-config)
- **Wrapper**: retry/cache/route/guardrail

Điểm = 1 con số

- correctness + LLM-eval quality
- latency / cost / error / drift ↓
- + thưởng theo chẩn đoán

Đội, ∼4h. **Public** test (giờ 2, leaderboard) → **private** (3.5h, held-out + 1 bug ẩn) xếp hạng. Nộp qua **git push**; model tự do (mock / local / cloud).

7 Anti-patterns Từ Industry

1. **“We’ll add monitoring later”** — later = never. Add ngay từ MVP.
2. **Log full prompts + responses** — vi phạm GDPR/PDPL, storage bill nổ. Sanitize + sample.
3. **Alert trên mọi metric “quan trọng”** — 50 alert $\rightarrow$ alert fatigue $\rightarrow$ ignore.
4. **Không có runbook** — alert fire 3h sáng, engineer trẻ lost, escalate lên senior.
5. **Monitoring dev $\neq$ prod config** — prod có issue không reproduce được vì dev khác setup.
6. **Chỉ đo performance, quên cost** — đến cuối tháng mới biết đốt tiền.
7. **Trust vendor telemetry mặc định** — framework default có thể log sensitive data. Đọc docs trước khi deploy.

Lưu ý: Anti-pattern #1 phổ biến và tai hại nhất. Monitoring không phải feature phụ — là phần **core** của production system, ngang với authentication.

Tổng kết — Key Takeaways

VINUNIVERSITY

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. **4 pillars.** Metrics + logs + traces + **eval** (còn đúng không). Chỉ logs là không đủ; AI cần pillar thứ 4.

2. **AI-specific metrics.** Token & cost (output đắt 5–6x input), P95 + TTFT, tool-call success. “HTTP 200” khác “trả lời đúng”.

3. **Logging + tracing.** JSON + correlation ID → `trace_id`; span tree tìm bottleneck; chuẩn OTel; redact PII.

4. **Alert + SLO + cost.** Page theo symptom & SLO burn, không theo cause. Đo cost như metric hạng nhất; cache để giảm.

5. **Online eval.** Sample → judge → gauge → alert. Debug incident: metric → log → trace.

Tiếp theo & Bài tập

AI Evaluation & Benchmarking

"Day 13 đo "chất lượng có còn đúng không" trên production. Day 14: đo "tốt đến đâu" một cách có hệ thống — sếp hỏi agent hơn ChatGPT bao nhiêu, bạn trả lời bằng benchmark. "

- Chuẩn bị: 10 câu hỏi mẫu + expected answer cho agent của bạn
- Đọc trước: tài liệu RAGAS (20 phút)
- Suy nghĩ: từ online eval hôm nay, quality metric nào quan trọng nhất cho use case của bạn?

Tài Liệu Tham Khảo

1. OpenTelemetry GenAI Semantic Conventions — `github.com/open-telemetry/semantic-conventions-genai` (trạng thái Development, 2026).
2. Langfuse — langfuse.com (OSS/MIT, SDK Python v4, OTel-based). LangSmith — docs.langchain.com.
3. Arize Phoenix & OpenInference; OpenLLMetry (Traceloop) — OTel-native LLM instrumentation.
4. Google SRE Book & SRE Workbook — sre.google (SLI/SLO/SLA, error budget, multi-burn-rate alerting, golden signals).
5. Prometheus & Grafana — prometheus.io, grafana.com. Microsoft Presidio — microsoft.github.io/presidio (PII redaction).
6. Vietnam PDPL (Luật 91/2025, hiệu lực 1/1/2026); Anthropic/OpenAI/Google pricing & prompt-caching docs.

Hỏi & Đáp

Monitoring tốt nghĩa là bạn biết agent có vấn đề trước khi user phàn nàn.